



Architectures of Convolutional Neural Networks

CS-4045: Deep Learning

Goals

- Today's Lecture
 - ConvNets vs. Traditional Computer Vision
 - ConvNet Architectures
 - LeNet
 - AlexNet
 - VGG
 - The Inception Family
 - V1, V2, V3, V4, Inception-ResNet, Xception
 - Resnet
 - DenseNet

ConvNets vs. Traditional for Computer Vision

- **Image classification** is the task of classifying a given image into one of the pre-defined categories.
- Traditional pipeline for image classification involves two modules:
 - **Feature extraction**
 - **Classification.**

Traditional Classification Pipeline

- **Feature extraction** involves extracting a higher level of information from raw pixel values that can capture the **distinction among the categories involved**.
- This feature extraction is done in an unsupervised manner wherein the **classes of the image have nothing to do with information extracted from pixels**.
- Some of the traditional and widely used features are **HOG, SIFT, LBP** etc.
- After the feature is extracted, a classification module is trained with the images and their associated labels. A few examples of this module are **SVM, Logistic Regression, Random Forest, decision trees** etc.

Traditional Classification Pipeline

- **The problem with this pipeline** is that feature extraction cannot be tweaked according to the **classes and images**.
- Features lack the representation required to distinguish the categories
 - **Accuracy of the classification** model suffers a lot, irrespective of the type of classification strategy employed.
- A common theme among the state of the art following the traditional pipeline has been, **to pick multiple feature extractors** and club them inventively to get a better feature.
- But this involves **too many heuristics as well as manual labor to tweak parameters according to the domain to reach a decent level of accuracy**.

Traditional Classification Pipeline

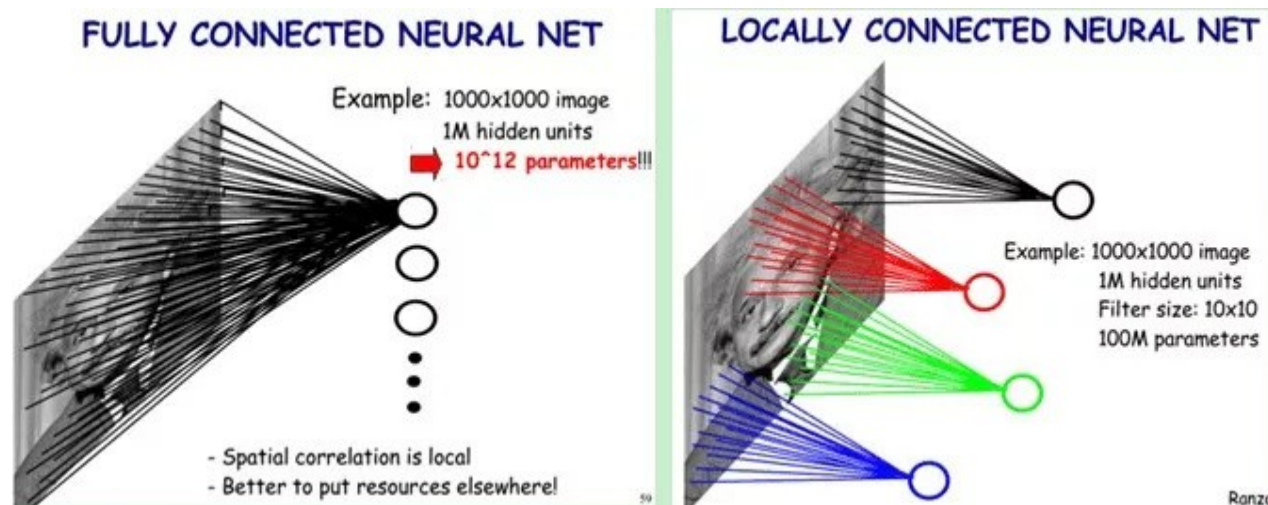
- **Another problem with this method** is that it is completely different from how **we humans learn to recognize things**.
- Just after birth, a child is incapable of perceiving his surroundings, but as he **progresses and processes data, he learns to identify things**.
- This is the philosophy behind **deep learning**, wherein **no hard-coded** feature extractor is built in.
- It combines the **extraction and classification modules into one integrated** system and it learns to extract, by discriminating representations from the images and classify them based on supervised data.

Automatic Feature Learning

- One such system is **multilayer perceptrons aka neural networks** which are multiple layers of neurons **densely connected** to each other.
- A deep vanilla neural network has such a large number of parameters involved that it is **impossible to train such a system without overfitting** the model due to the lack of a sufficient number of training examples.
- But with **Convolutional Neural Networks(ConvNets)**, the task of training the whole network from the scratch can be carried out using a large dataset like ImageNet.

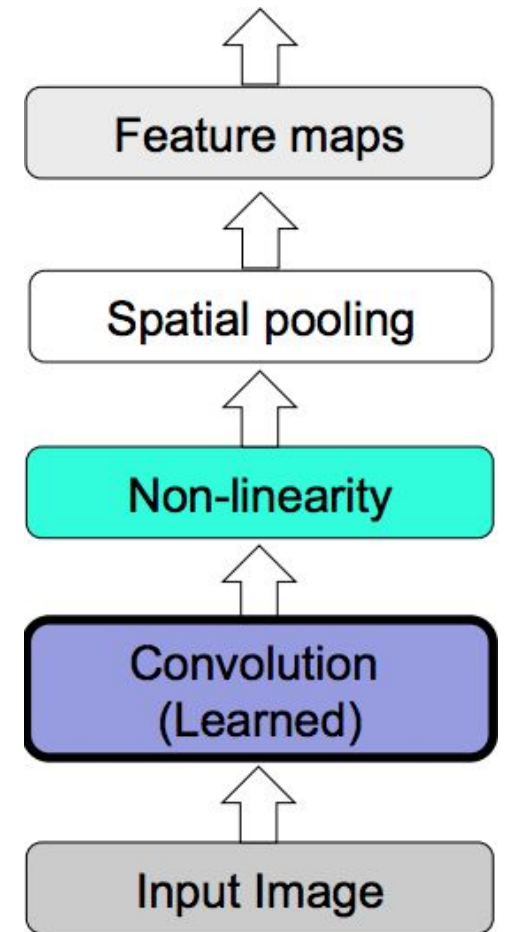
Automatic Feature Learning

- The reason behind this is, **sharing of parameters** between the neurons and **sparse connections in convolutional layers**.
- In the convolution operation, the neurons in one layer are only locally connected to the input neurons and the set of parameters are shared across the 2-D feature map.



ConvNet Architectures

- Almost all CNN architectures follow the same general design principles of successively applying **convolutional layers** to the input.
- Periodically **downsampling the spatial dimensions** while increasing the **number of feature maps**.



ConvNet Architectures

- While the classic network architectures were comprised simply of **stacked convolutional layers**, modern architectures explore new and innovative ways for constructing convolutional layers in a way which allows for **more efficient learning**.
- Almost all of these architectures are based on a **repeatable unit** which is used throughout the network.

LetNet-1998

Gradient-based learning applied to document recognition

[Y LeCun](#), [L Bottou](#), [Y Bengio](#), [P Haffner](#)

Proceedings of the IEEE, 1998 • ieeexplore.ieee.org

Multilayer neural networks trained with the back-propagation algorithm constitute the best example of a successful gradient based learning technique. Given an appropriate network architecture, gradient-based learning algorithms can be used to synthesize a complex decision surface that can classify high-dimensional patterns, such as handwritten characters, with minimal preprocessing. This paper reviews various methods applied to handwritten character recognition and compares them on a standard handwritten digit

SHOW MORE ▾

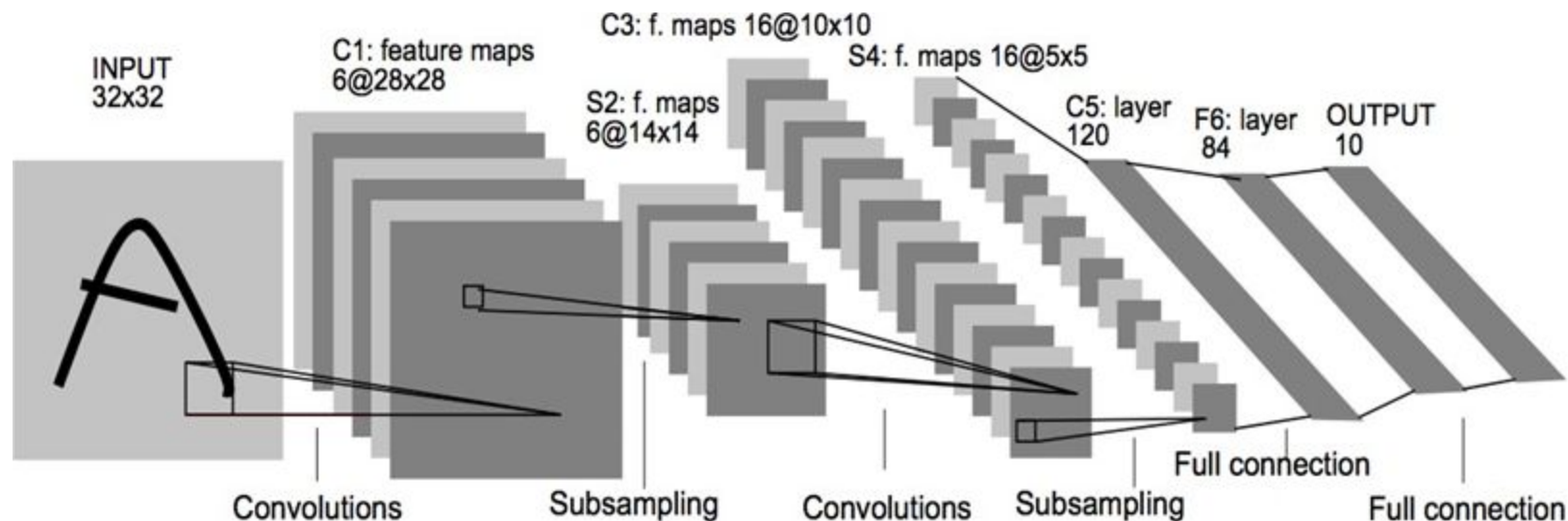
☆ Save  Cite Cited by 73344 Related articles All 37 versions

Gradient-Based Learning Applied to Document Recognition

Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner

LeNet-5

- Yann Lecun's **LeNet-5 model** was developed in 1998 to identify handwritten digits for zip code recognition in the postal service.
- This pioneering model largely introduced the **convolutional neural network** as we know it today.



LeNet-5

- Convolutional layers use a subset of the previous layer's channels for each filter to reduce computation and force a break of symmetry in the network.
- The subsampling layers use a form of **average pooling**.
- **Parameters:** 60,000
- **Paper:** [Gradient-based learning applied to document recognition](#)

LeNet-5: Reading a Check



Architecture of LeNet-5

1. Input Layer (32x32 Image)

- The input to LeNet is a **32×32 grayscale image** (e.g., MNIST digits).
- Since MNIST images are 28x28, padding is applied to make them 32x32.

2. First Convolutional Layer (C1)

- **6 filters of size 5×5** (stride = 1).
- The resulting feature maps have size **28×28×6**.
- **Activation Function:** Sigmoid (original) or ReLU (modern version).

3. First Subsampling Layer (S2 - Pooling)

- **Average Pooling** with a **2×2** filter and stride 2.
- Output size is **14×14×6**.
- Pooling helps reduce spatial dimensions while retaining important information.

Architecture of LeNet-5

4. Second Convolutional Layer (C3)

- **16 filters of size 5×5** (some filters are connected to only a subset of previous feature maps to introduce sparsity).
- The output is $10 \times 10 \times 16$.
- Again, activation is Sigmoid/ReLU.

5. Second Subsampling Layer (S4 - Pooling)

- **Average Pooling** with a 2×2 filter.
- Output size is $5 \times 5 \times 16$.

6. Fully Connected Layer (C5)

- Flattened input from previous layer ($5 \times 5 \times 16 = 400$ neurons).
- Fully connected to **120 neurons**.
- Activation: Sigmoid/ReLU.

Architecture of LeNet-5

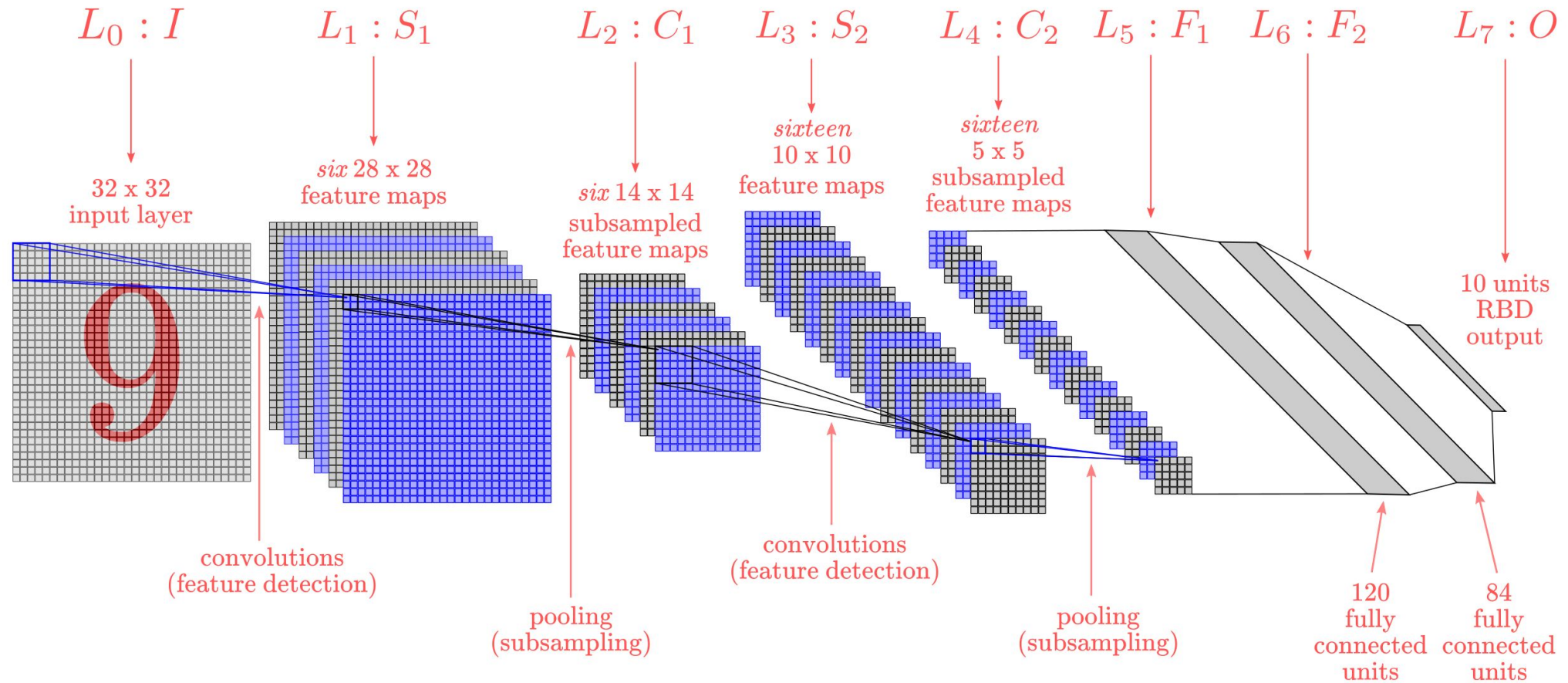
7. Fully Connected Layer (F6)

- Fully connected to **84 neurons**.
- Activation: Sigmoid/ReLU.

8. Output Layer

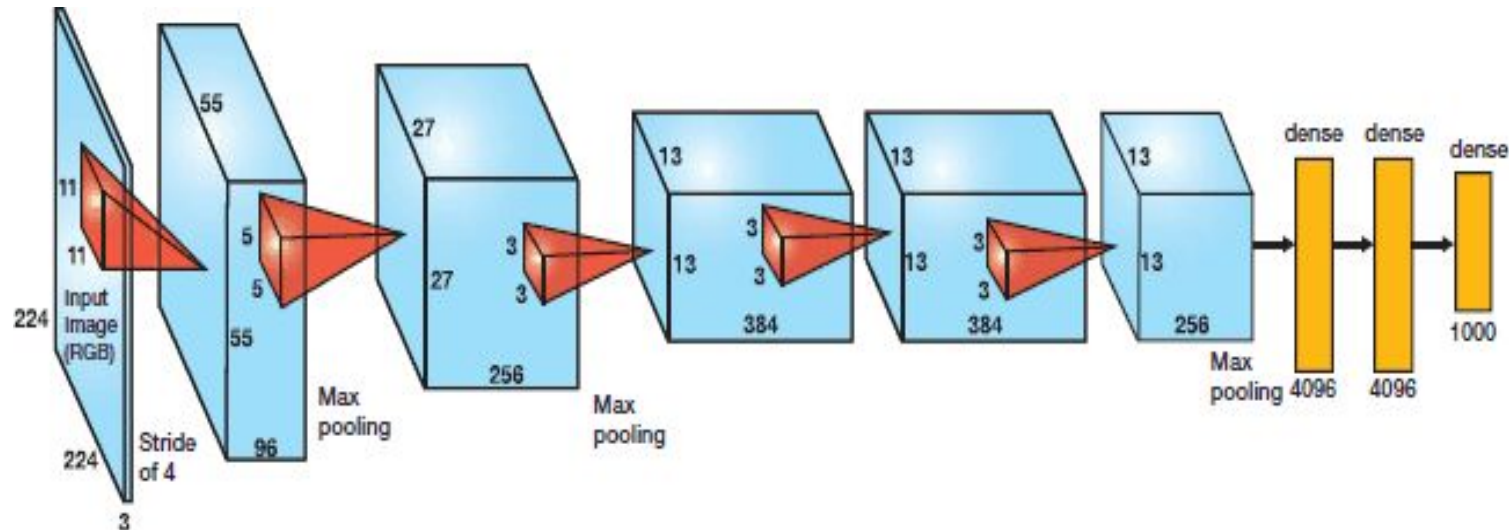
- Fully connected to **10 output neurons** (for digit classification 0-9).
- Activation: **Softmax** (for multi-class classification).

Architecture of LeNet-5



AlexNet

- AlexNet was developed by Alex Krizhevsky et al. in 2012 to compete in the **ImageNet** competition.
- The general architecture is quite similar to LeNet-5, although this model is **considerably larger**.



AlexNet

- [Download Paper](#)

Imagenet classification with deep convolutional neural networks

[A Krizhevsky, I Sutskever...](#) - Advances in neural ..., 2012 - [proceedings.neurips.cc](#)

We trained a large, deep convolutional neural network to classify the 1.3 million high-resolution images in the LSVRC-2010 ImageNet training set into the 1000 different classes. On the test data, we achieved top-1 and top-5 error rates of 39.7\% and 18.9\% which is considerably better than the previous state-of-the-art results. The neural network, which has 60 million parameters and 500,000 neurons, consists of five convolutional layers, some of which are followed by max-pooling layers, and two globally connected layers with a final ...

☆ Save  Cite Cited by 140813 Related articles All 89 versions 

AlexNet

ImageNet Classification with Deep Convolutional Neural Networks

Alex Krizhevsky
University of Toronto
kriz@cs.utoronto.ca

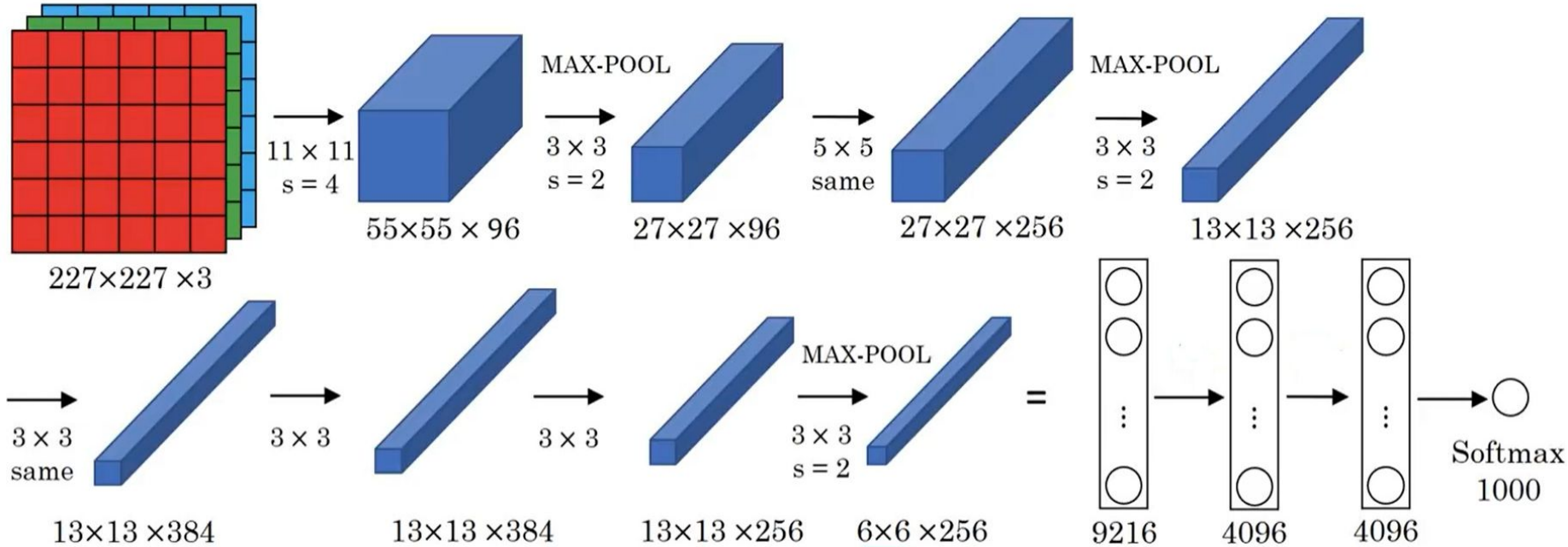
Ilya Sutskever
University of Toronto
ilya@cs.utoronto.ca

Geoffrey E. Hinton
University of Toronto
hinton@cs.utoronto.ca

Abstract

We trained a large, deep convolutional neural network to classify the 1.2 million high-resolution images in the ImageNet LSVRC-2010 contest into the 1000 different classes. On the test data, we achieved top-1 and top-5 error rates of 37.5% and 17.0% which is considerably better than the previous state-of-the-art. The neural network, which has 60 million parameters and 650,000 neurons, consists of five convolutional layers, some of which are followed by max-pooling layers, and three fully-connected layers with a final 1000-way softmax. To make training faster, we used non-saturating neurons and a very efficient GPU implementation of the convolution operation. To reduce overfitting in the fully-connected layers we employed a recently-developed regularization method called “dropout” that proved to be very effective. We also entered a variant of this model in the ILSVRC-2012 competition and achieved a winning top-5 test error rate of 15.3%, compared to 26.2% achieved by the second-best entry.

AlexNet



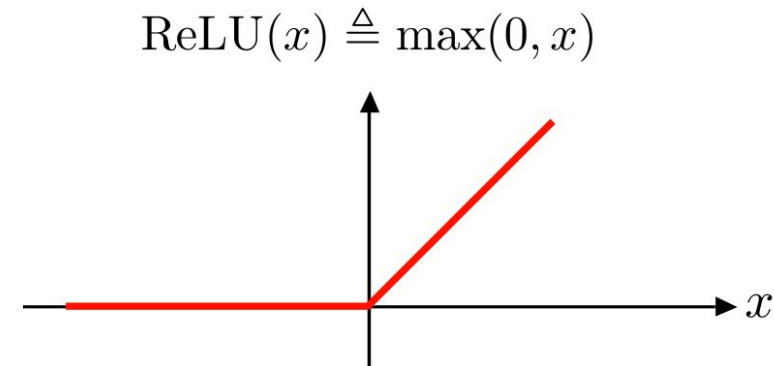
Similarities with LeNet but much bigger

AlexNet

- It starts with $227 \times 227 \times 3$ images and the next convolution layer **applies 96 of 11×11 filter** with stride of 4.
- The output volume reduce its dimension by **55×55** .
- Next layer is a **pooling layer** which applies max pool by 3×3 filter along with stride 2.
- It goes on and finally reaches **FC layer with 9216 parameter** and the next two **FC layers with 4096 node each**.
- At the end, it uses **Softmax function** with 1000 output classes.
- Parameters: 60 Million.

AlexNet

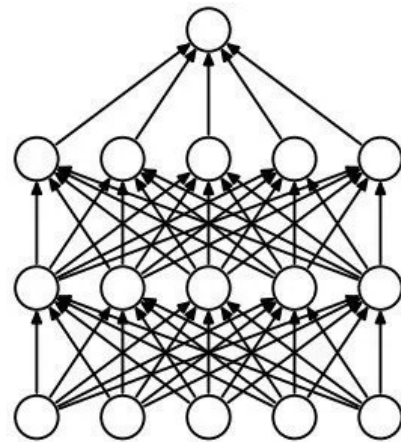
- It uses **ReLU activation function** instead Sigmoid or tanh functions (In LeNet).
- **The advantage of the ReLU over sigmoid is that it trains much faster** than the latter because the **derivative of sigmoid becomes very small** in the saturating region and therefore the updates to the weights almost vanish
- It speed by more than 5 times faster with same accuracy



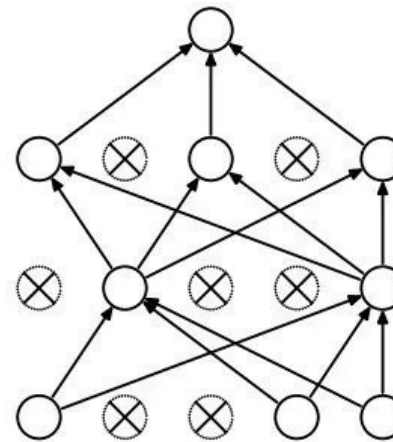
Dying ReLU problem: If a neuron is inactive ($x \leq 0$), its gradient becomes zero

AlexNet

- Another problem that this architecture solved was reducing the **over-fitting** by using a **Dropout layer after every FC layer**.
- **Dropout layer has a probability, (p)**, associated with it and is applied at every neuron of the response map separately. It randomly switches off the activation with the probability p



(a) Standard Neural Net



(b) After applying dropout.

VGG

Very deep convolutional networks for large-scale image recognition

[K Simonyan](#), [A Zisserman](#) - arXiv preprint arXiv:1409.1556, 2014 - arxiv.org

In this work we investigate the effect of the convolutional network depth on its accuracy in the large-scale image recognition setting. Our main contribution is a thorough evaluation of networks of increasing depth using an architecture with very small (3x3) convolution filters, which shows that a significant improvement on the prior-art configurations can be achieved by pushing the depth to 16-19 weight layers. These findings were the basis of our ImageNet Challenge 2014 submission, where our team secured the first and the second places in the ...

☆ Save  Cite Cited by 110827 Related articles All 43 versions 

VGG

VERY DEEP CONVOLUTIONAL NETWORKS FOR LARGE-SCALE IMAGE RECOGNITION

Karen Simonyan* & Andrew Zisserman*

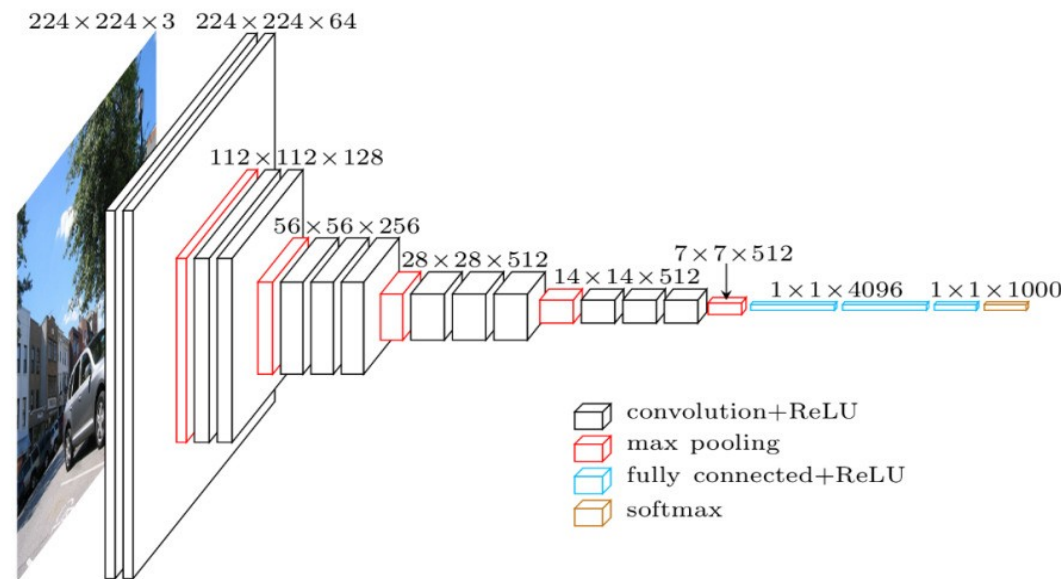
Visual Geometry Group, Department of Engineering Science, University of Oxford
{karen, az}@robots.ox.ac.uk

ABSTRACT

In this work we investigate the effect of the convolutional network depth on its accuracy in the large-scale image recognition setting. Our main contribution is a thorough evaluation of networks of increasing depth using an architecture with very small (3×3) convolution filters, which shows that a significant improvement on the prior-art configurations can be achieved by pushing the depth to 16–19 weight layers. These findings were the basis of our ImageNet Challenge 2014 submission, where our team secured the first and the second places in the localisation and classification tracks respectively. We also show that our representations generalise well to other datasets, where they achieve state-of-the-art results. We have made our two best-performing ConvNet models publicly available to facilitate further research on the use of deep visual representations in computer vision.

VGG

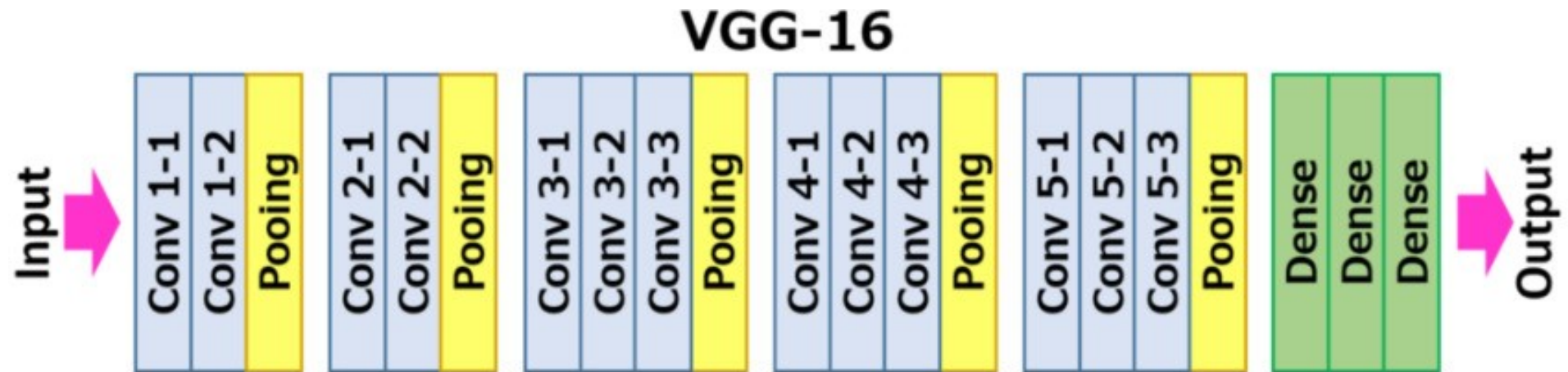
- This architecture is from **VGG group, Oxford**.
- The VGG network, introduced in 2014, offers a deeper yet simpler variant of the convolutional structures.
- At the time of its introduction, this model was considered to be very deep.



VGG

- It makes the improvement over AlexNet by *replacing large kernel-sized filters* (11 and 5 in the first and second convolutional layer, respectively) with *multiple 3X3 kernel-sized filters* one after another.
- With a given *receptive field* (the effective area size of input image on which output depends), *multiple stacked smaller size kernel is better* than the one with a larger size kernel.
 - Multiple non-linear layers increases the *depth of the network which enables it to learn more complex features*, and that too at a lower cost.
- It always uses 3 x 3 filters with stride of 1 in convolution layer and uses SAME padding in pooling layers 2 x 2 with stride of 2 – *Simplified architecture*

VGG



Parameters: 138 million

VGG

```
1 from keras.models import Sequential
2 from keras.layers import Dense, Activation, Dropout, Flatten
3 from keras.layers import Conv2D
4 from keras.layers import MaxPooling2D
5
6 input_shape = (224, 224, 3)
7
8 model = Sequential([
9     Conv2D(64, (3, 3), input_shape=input_shape, padding='same',
10         activation='relu'),
11     Conv2D(64, (3, 3), activation='relu', padding='same'),
12     MaxPooling2D(pool_size=(2, 2), strides=(2, 2)),
13     Conv2D(128, (3, 3), activation='relu', padding='same'),
14     Conv2D(128, (3, 3), activation='relu', padding='same'),
15     MaxPooling2D(pool_size=(2, 2), strides=(2, 2)),
16     Conv2D(256, (3, 3), activation='relu', padding='same'),
17     Conv2D(256, (3, 3), activation='relu', padding='same'),
18     Conv2D(256, (3, 3), activation='relu', padding='same'),
19     MaxPooling2D(pool_size=(2, 2), strides=(2, 2)),
20     Conv2D(512, (3, 3), activation='relu', padding='same'),
21     Conv2D(512, (3, 3), activation='relu', padding='same'),
22     Conv2D(512, (3, 3), activation='relu', padding='same'),
23     MaxPooling2D(pool_size=(2, 2), strides=(2, 2)),
```

```
24     Conv2D(512, (3, 3), activation='relu', padding='same'),
25     Conv2D(512, (3, 3), activation='relu', padding='same'),
26     Conv2D(512, (3, 3), activation='relu', padding='same'),
27     MaxPooling2D(pool_size=(2, 2), strides=(2, 2)),
28     Flatten(),
29     Dense(4096, activation='relu'),
30     Dense(4096, activation='relu'),
31     Dense(1000, activation='softmax')
32 ])
33
34 model.summary()
```

```
from keras.applications import VGG16, VGG19
VGG16.summary()
VGG19.summary()
```


Pre-Trained Models

```
import torch
import torchvision.models as models

# Load pretrained AlexNet
alexnet = models.alexnet(pretrained=True)

# Load pretrained VGG16
vgg16 = models.vgg16(pretrained=True)

# Load pretrained VGG19
vgg19 = models.vgg19(pretrained=True)
```

```
from tensorflow.keras.applications import VGG16, VGG19

# Load VGG16 with pretrained ImageNet weights
vgg16 = VGG16(weights="imagenet")

# Load VGG19 with pretrained ImageNet weights
vgg19 = VGG19(weights="imagenet")
```

What is Transfer Learning?

- Transfer learning is a machine learning technique where a model trained on a large dataset (e.g., ImageNet) is reused and fine-tuned on a smaller, specific dataset.
- **Avoids Training from Scratch** – Training deep models like VGG or AlexNet from scratch on small datasets is inefficient.
- **Reduces Training Time** – Since the model already knows basic patterns (edges, textures, shapes), only the last layers need to be trained.

What is Transfer Learning?

- **Prevents Overfitting** – With fewer data points, training a deep network from scratch can lead to overfitting.
 - Transfer learning mitigates this.
- **Improves Accuracy** – Pretrained models on large datasets generalize better and learn more effectively on smaller datasets.

When to Use Transfer Learning?

- Recommended Scenarios:
 - Small dataset (few thousand images)
 - Similar dataset to ImageNet (e.g., Cars, Animals, Fruits)
- Not Recommended
 - Completely different dataset (e.g., Medical Images, X-rays)
 - Large dataset (>1M images)

How to Use Transfer Learning?

- There are **two main approaches**:
- **A. Feature Extraction (Frozen Layers)**
 - Keep **pretrained convolutional layers frozen** (don't update weights).
 - Replace the **fully connected layers** and train only them.
 - **Used when dataset is small but similar to the original dataset.**
- **B. Fine-tuning (Unfreeze Some Layers)**
 - Unfreeze some **deep convolutional layers** and train them along with the new classifier.
 - **Used when the dataset is large or very different** from ImageNet

Using VGG16 for a Custom Dataset

- Load VGG16 pretrained on ImageNet (**1000 classes**).
- Remove the last layers (fully connected layers).
- Add custom layers for our dataset (**e.g., 10 classes**).
- Train on a small dataset.

Step 1: Load Pretrained VGG16 Model

```
import tensorflow as tf
from tensorflow.keras.applications import VGG16
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Dense, Flatten, Dropout
from tensorflow.keras.preprocessing.image import ImageDataGenerator

# Load VGG16 without top layers (fully connected layers)
base_model = VGG16(weights="imagenet", include_top=False, input_shape=(224, 224, 3))

# Freeze convolutional base
base_model.trainable = False
```

Step 2: Add Custom Layers

```
# Add new fully connected layers
x = Flatten()(base_model.output)
x = Dense(256, activation='relu')(x)
x = Dropout(0.5)(x) # Regularization to prevent overfitting
x = Dense(10, activation='softmax') # 10 classes in our new dataset

# Create new model
model = Model(inputs=base_model.input, outputs=x)
```

Step 3: Compile and Train

```
# Compile model
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

# Prepare Data (Example: Small Custom Dataset)
train_datagen = ImageDataGenerator(rescale=1./255, validation_split=0.2)

✓ train_generator = train_datagen.flow_from_directory(
    "c:/dataset/Animals",
    target_size=(224, 224),
    batch_size=32,
    class_mode='categorical',
    subset='training'
)

✓ val_generator = train_datagen.flow_from_directory(
    "c:/dataset/Animals",
    target_size=(224, 224),
    batch_size=32,
    class_mode='categorical',
    subset='validation'
)

# Train the model
model.fit(train_generator, validation_data=val_generator, epochs=10)
```

If we set class_mode='sparse', labels are assigned as integers
Cat → 0
Dog → 1
Elephant → 2

Fine-Tuning

```
# Unfreeze last 3 layers for fine-tuning
for layer in base_model.layers[-3:]:
    layer.trainable = True

# Recompile with a lower learning rate
model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=0.0001),
              loss='categorical_crossentropy',
              metrics=['accuracy'])

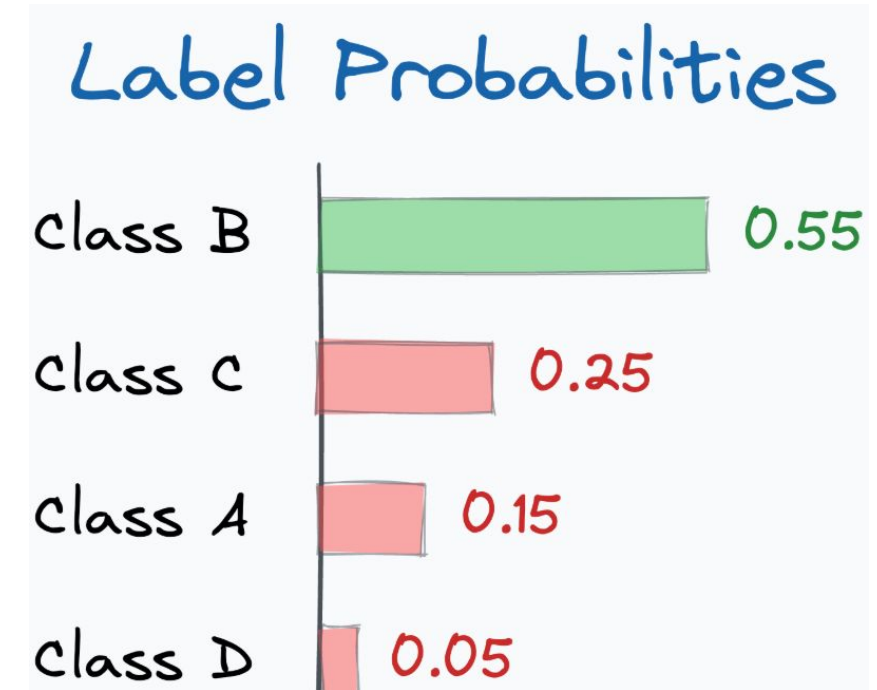
# Continue training
model.fit(train_generator, validation_data=val_generator, epochs=10)
```

Evaluation Metrics

- Accuracy is a common metric for classification models.

$$\text{Accuracy} = \frac{\text{Correct Predictions}}{\text{Total Predictions}}$$

- **Top-K Categorical Accuracy** (for large class datasets, like ImageNet):
 - Checks if the correct label is in the top K predictions.



Evaluation Metrics

- Precision and Recall (for imbalanced data):

Precision measures how many of the predicted "positive" cases were actually correct.

📌 Formula:

$$\text{Precision} = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Positives (FP)}}$$

- True Positives (TP): Cases correctly predicted as positive.
- False Positives (FP): Cases incorrectly predicted as positive.
- ♦ High Precision → Fewer False Positives

Evaluation Metrics

Recall measures how many of the actual positive cases were correctly identified.

▶ Formula:

$$\text{Recall} = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Negatives (FN)}}$$

- **False Negatives (FN):** Cases that should have been positive but were misclassified as negative.
- **High Recall → Fewer False Negatives**
- If Recall is **high**, the model is good at **finding all positive cases**.

✦ Example - Medical Diagnosis (Cancer Detection)

Scenario	Precision	Recall
Doctors diagnosing cancer	⚠️ Low Precision (many non-cancer cases falsely diagnosed)	✅ High Recall (almost all real cancer cases detected)
Spam detection	✅ High Precision (only real spam is marked)	⚠️ Low Recall (some spam emails go undetected)

Thank You 😊